NMAM INSTITUTE OF TECHNOLOGY,NITTE

Department of Information Science and Engineering

OPERATING SYSTEM PROGRAMMING ASSIGNMENT REPORT

Name: Diya P Shetty

USN: NNM24IS080

Course: Operating Systems

GitHub Repository: https://github.com/DiyaPShetty/OS_Task2

SUBMITTED TO

Dr Jason Elroy Martis

Associate Professor

Dept Information Science and Engineering ,nitte

Abstract

This coursework presents the design and implementation of two classic process-synchronization problems in Operating Systems: the Dining Philosophers problem and the Readers-Writers problem. Each problem is implemented in Java using two techniques, semaphores and monitors, which gives four Java programs. The Dining Philosophers programs use synchronization to avoid race conditions while several competing threads share the chopsticks.

The Readers Writers problem allows more than one reader to have access to common data simultaneously but gives exclusive access to the writer.

Each Java program keeps track of what happens while it runs. This information is then used to create an HTML animation, where we can see how the threads work, how they access shared resources, and how synchronization controls their activities. This report explains how the programs work and compares the advantages and disadvantages of using semaphores and monitors for thread synchronization.

1. Introduction

An operating system manages many processes, and each process can have several threads. In some cases, multiple threads access the same information or use the same resources while the program is running. If multiple threads read and modify shared data in an uncontrolled order, the program output becomes unpredictable. Such a situation is known as race condition [2].Synchronization techniques are used to prevent thread from modifying same data simultaneously.

This assignment shows thread synchronization using semaphores and monitors. The programs use Java threads and log events such as waiting, entering and leaving critical sections, and updating shared data. These logs are then used to generate HTML pages that can be viewed in a web browser.

2. Objectives

- Implement Readers-Writers using semaphores.
- Implement Readers-Writers using a monitor-style design.
- Implement Dining Philosophers with four philosophers using semaphores.
- Implement Dining Philosophers with four philosophers using a monitor.
- Record events while the programs run and use the recorded information to create HTML animations that display the execution process in a web browser.
- Compare the semaphore and monitor methods, and document the testing and AI assistance.

3. Development Environment

All programs in this assignment use Java threads to run multiple tasks at the same time. The programs that use semaphores rely on Java's Semaphore class, which is available in the java.util.concurrent package.

The monitor-based programs use the synchronized keyword together with the wait() and notifyAll() methods where needed.

The programs log events during execution and generate HTML files that use CSS and JavaScript. The HTML files show thread activity, shared-resource access, and synchronization behaviour. They are opened in a web browser for visualization.

Environment details:

- Operating System: Windows 11
- Programming Language: Java
- Java Version: Java 23.0.1
- Editor/IDE: Visual Studio Code
- Terminal: Windows PowerShell
- Web Browser: Google Chrome

4. Problem 1: Dining Philosophers Using Semaphores (qstn1.java)

4.1 Problem Statement

Dining Philosophers is a classic synchronization problem in Operating Systems. This program simulates philosophers sitting at a table. Four philosophers take turns thinking and eating.Each philosopher has two chopsticks nearby, shared with the philosophers sitting on either side. To start eating, a philosopher must get hold of both chopsticks. A chopstick cannot be used by two philosophers at the same time.

If every philosopher picks up one chopstick and then waits for the other, no one can continue. This situation is called deadlock. In that scenario, each philosopher is given one chopstick and then waits for the other chopstick from his neighbor, which he will never get.

4.2 Synchronization Design

Java threads represent the philosophers, while semaphores manage access to the chopsticks and help prevent multiple threads from using the same chopstick simultaneously. Four philosopher threads run at the same time, and each philosopher eats three meals.

The synchronization design uses the following parts:

- Four semaphores control access to the four chopsticks, ensuring that each chopstick is used by only one philosopher at a time.
- **Mutex semaphore:** A semaphore with one permit. It protects shared data such as philosopher status, chopstick ownership, the waiting queue, and the event records.
- There is a distinct semaphore per philosopher that begins at zero permits.A philosopher waits on its individual semaphore until the operating system allows it to proceed.
- **FIFO waiting queue:** Philosophers are placed in a queue when they become hungry, and they are considered in the order they joined.The philosopher who has been waiting the longest gets priority.The scheduler checks the philosopher at the front of the queue to see whether both of its chopsticks are available.
- **HTML simulation:** The program logs changes in philosopher states, chopstick ownership, and queue positions. These logs are used to generate an HTML simulation that shows the philosophers, chopsticks, queue, and execution timeline.
-

4.2.1 Deadlock Prevention

The program uses a FIFO scheduling method. When a philosopher becomes hungry, the philosopher is added to the waiting queue. The scheduler then uses tryAcquire() to test whether both chopsticks are available.

If the philosopher can get both chopsticks, the philosopher eats. If the philosopher can get the first chopstick but not the second, the program releases the first chopstick instead of holding it while waiting for the second.

This prevents a philosopher from holding one chopstick while waiting indefinitely for another. This means that it bypasses the hold-and-wait situation, which is among the four requirements for deadlock [2].

4.3 Expected Observations and Testing

During execution, each philosopher moves through the states thinking, hungry, and eating, and then releases the chopsticks. A philosopher who becomes hungry joins the FIFO queue and waits until the scheduler allows it to use both chopsticks.

After eating, the philosopher releases both chopsticks so that another waiting philosopher can use them.

The event log records the states, the chopstick holders, and the queue. The HTML simulation has Play, Pause, and Reset buttons for watching the recorded sequence.

Output file: dining_philosophers_semaphore.html

Observed result: The program completed, and each of the four philosophers ate three meals. The HTML simulation file dining_philosophers_semaphore.html was created. The event trace showed the philosophers changing between the thinking, hungry, and eating states. It also showed that each chopstick was assigned to only one philosopher at a time, and that a philosopher waited until a chopstick was available before picking it up.

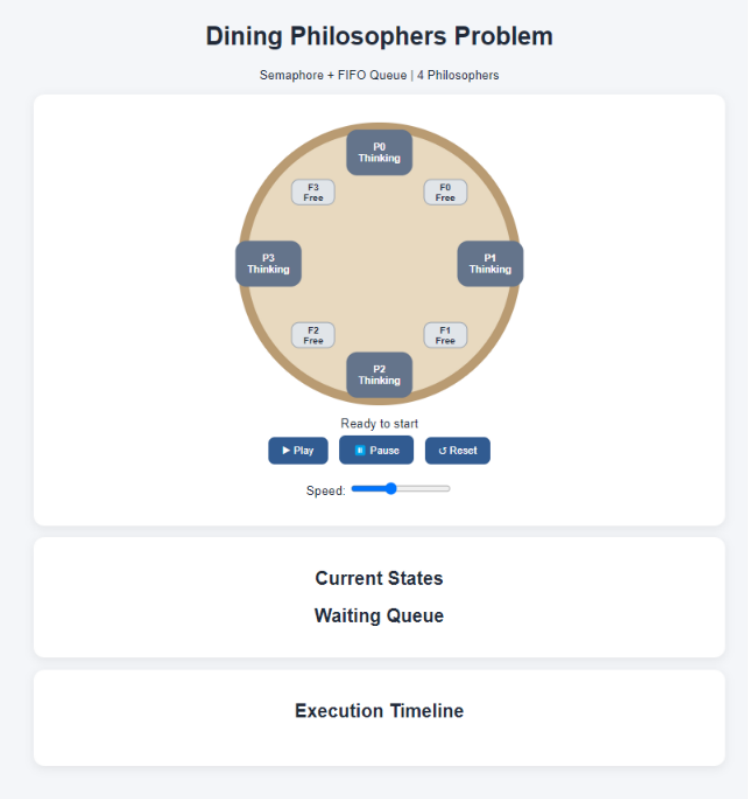


Figure 1: Dining Philosophers: Semaphore HTML Simulation

5. Problem 2: Dining Philosophers Using a Monitor (qstn2.java)

5.1 Monitor Concept

A monitor is a synchronization construct that combines shared data with the code that accesses that data. It allows only one thread at a time to execute the synchronized code of an object.

In Java, the synchronized keyword provides mutual exclusion using the object's lock. The wait() method releases the lock and puts the thread into a waiting state. The notifyAll() operation will awaken all waiting threads and each of those threads will check the condition once more.again [2], [7]–[9].

5.2 Synchronization Design

Monitor keeps track of the state of each philosopher and the usage of the four chopsticks. The philosopher makes a request for permission to eat by making a call to a synchronized method.The monitor checks whether the neighbouring philosophers are eating. If one of them is, the philosopher waits until the monitor notifies it to check again.

Once a philosopher finishes eating, the program updates the shared state and signals the other threads that are waiting.

The main parts of the synchronization design are:

- **Mutual exclusion:** The synchronized keyword protects the shared data from being changed by more than one thread at a time.
- **State check:** After wait() returns, the thread checks the state again.
- **Thread notification:** notifyAll() tells the waiting threads to check whether they can continue.

The HTML simulation shows the philosopher states and the chopstick usage during execution.

5.3 Expected Results and Testing

This program should show that monitor-based synchronization prevents two philosophers from using the same chopstick at the same time. The event trace can be used to check that two neighbouring philosophers never eat at the same time, and that waiting philosophers continue once the chopsticks are available.

While the program runs, check the following:

- The philosophers move through the states thinking, hungry, and eating.
- Neighbouring philosophers do not eat at the same time.
- Waiting philosophers continue after they are notified.

The HTML simulation was generated, and Figure 2 shows its interface.

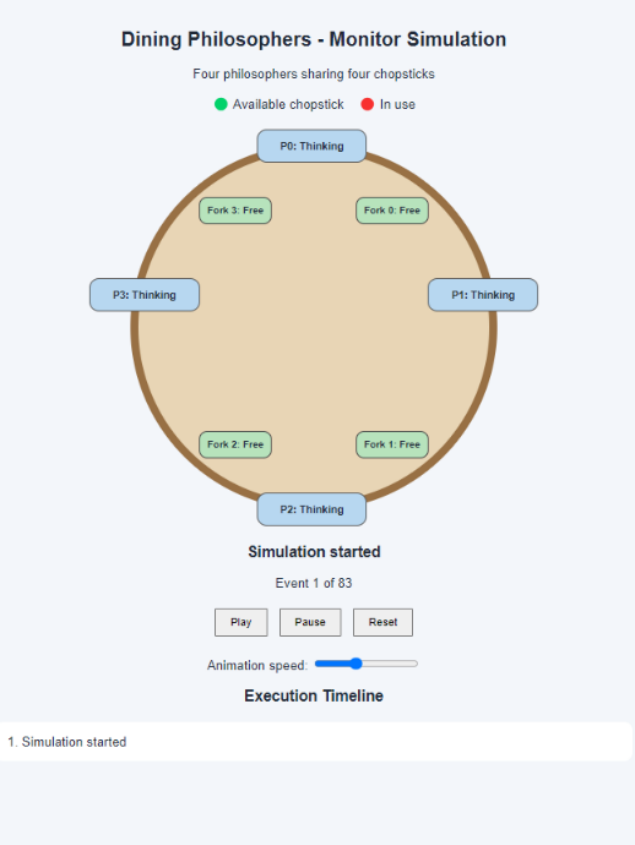


Figure 2: Dining Philosophers: Monitor HTML Simulation

6. Problem 3: Readers-Writers Using Semaphores (qstn3.java)

6.1 Problem Statement

The Readers-Writers problem is about how several threads can safely access shared data. Multiple readers can read the data at the same time because reading does not change it. A writer needs exclusive access because it changes the data. This setup avoids clashes while using common data. Readers cannot read while a writer is updating the data, and only one writer can make changes at a time [2], [6].

6.2 Semaphore Design

The program uses semaphores to manage the readers and writers. One semaphore protects the reader count, and another acts as a write lock for the shared data.

When the first reader starts reading, it locks the data against writers. Other readers can still read the same data at the same time, but writers must wait until all readers have finished.The last reader to finish releases the write lock. A writer acquires the write lock before updating the data and releases it afterwards, so that other threads can continue.

The main features of the implementation are:

- **Mutex semaphore:** Protects updates to readerCount.
- **Write-lock semaphore:** Blocks writers while readers are reading, and allows only one writer at a time.
- **Shared data:** Writers change the data, and readers only read it.
- **HTML visualization:** The HTML page displays the events recorded during execution, including reader and writer activities, thread states, the current reader count, whether a writer is active, and the value stored in shared data.
-

6.3 Limitations

This implementation gives preference to readers. New readers can enter while other readers are already in the critical section. If readers keep arriving, a waiting writer may never get the write lock, so the writer may starve [2], [3].

6.4 Results

This program shows how semaphores can control access to shared data. The event trace can be verified to ensure that multiple readers are reading concurrently, that writers have exclusive write access, and that the shared value is updated whenever a writer modifies it.

The HTML simulation shows the reader and writer states, the reader count, the writing activity, and changes to the shared data.



Figure 3: Readers-Writers: Semaphore HTML Simulation

7. Problem 4: Readers-Writers Using a Monitor (qstn4.java)

7.1 Monitor Design

In this algorithm, a Java monitor is used for controlling the access of readers and writers to the shared resource. It. It ensures that they follow the required synchronization rules while working with the data.Synchronized methods protect the shared data, and wait() and notifyAll() let threads communicate when the resource is not available.

The monitor keeps track of the number of active readers, whether a writer is active, and the number of waiting writers. A reader has to wait if a writer is writing or another writer is waiting to write. Similarly, a writer can proceed only when no other writer is active and no readers are accessing the shared data.When a reader or writer leaves, the monitor updates the shared state and calls notifyAll(). The waiting threads then check their conditions again.

This implementation gives priority to waiting writers to prevent writer starvation.

7.2 Generated HTML Dashboard

When the Java threads finish, the program generates an HTML file that shows the sequence of events. It consists of the summary, status of the reader and the writer, status of the shared object, and the list of events.

The dashboard has the following controls:

- **Play:** Moves through the recorded events automatically.
- **Pause:** Stops the automatic playback.
- **Next Event:** Moves to the next recorded event.
- **Reset:** Returns the visualization to the beginning.
- **Speed:** Changes how fast the events are shown.

These controls only replay the recorded events in the browser.

7.3 Observed Result

This program uses a Java monitor to coordinate readers and writers when accessing shared data. Multiple readers can read the data simultaneously, provided that no writer is currently writing to it.When a writer wants to write, it needs exclusive access.

In the test run, the four reader threads and the two writer threads each completed three iterations. The shared data value rose from 0 to 6 as the writers updated it. The program also generated the HTML dashboard readers_writers_monitor.html, which can be opened in a browser to view the recorded events.

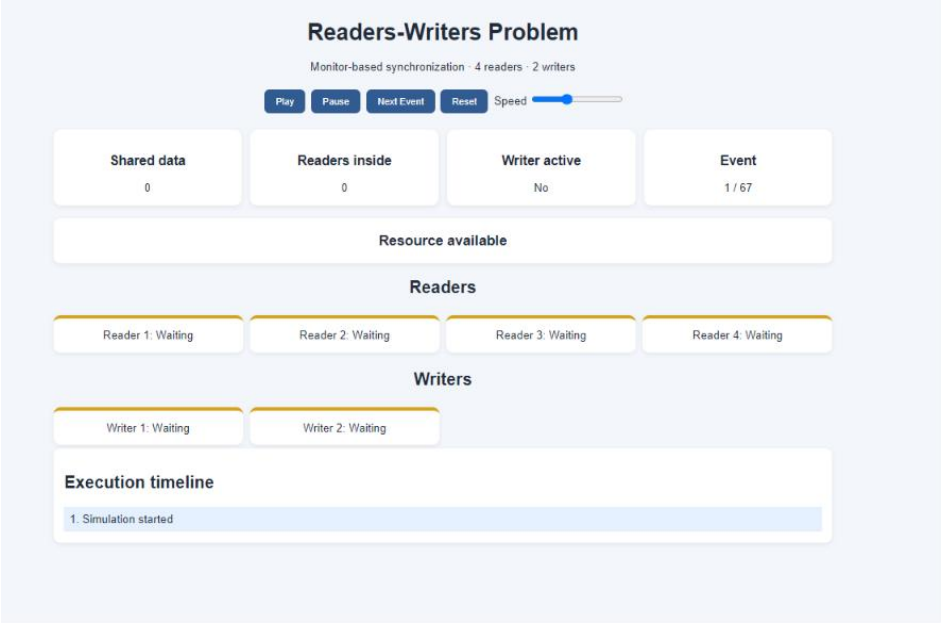


Figure 4: Readers-Writers: Monitor HTML Simulation

8. HTML Generation Workflow

The HTML files are generated from events that the Java program records while it runs.

The process works as follows:

1. Run the Java program.
2. The program creates the required philosopher, reader, or writer threads.
3. The threads are synchronized using semaphores or monitors, depending on the program.
4. The program monitors changes in thread state and shared resources as events.
5. After the threads finish, the program generates an HTML file from the recorded events.
6. Open the HTML file in a web browser to view the event timeline or dashboard.

Because the HTML visualization is generated from the recorded data, it shows how the threads interacted and competed for resources during execution.

9. Execution and Build Instructions

The programs need the Java Development Kit (JDK) to compile and run. Open the terminal window in the directory that contains your Java code, then type the following commands to compile and execute the program.

Program 1: Dining Philosophers Using Semaphores

```
javac qstn1.java
java qstn1
```

Program 2: Dining Philosophers Using a Monitor

```
javac qstn2.java
java qstn2
```

Program 3: Readers-Writers Using Semaphores

```
javac qstn3.java
java qstn3
```

Program 4: Readers-Writers Using a Monitor

```
javac qstn4.java
java qstn4
```

Run each program separately, check the terminal output for the name of the generated HTML file, and open that file in a web browser to view the visualization.

If you change the Java source code, compile and run the program again to generate an updated HTML file.

10. Concepts and Correctness

10.1 Race Condition

A race condition happens when the result of a program depends on the order in which threads access shared data [2]. In these programs, the shared data includes the reader count, the writer status, the philosopher states, the chopstick owners, and the shared data value.

Semaphores and Java monitors are used to stop threads from making conflicting changes to this data. For instance, the mutex semaphore protects the reader count in the semaphore-based Readers-Writers algorithm.

10.2 Mutual Exclusion

Mutual exclusion means that only one thread can use a resource at a time [2]. In the Readers-Writers problem, a writer needs mutual exclusion: it cannot write while any reader or other writer is using the data. Multiple readers can read together as long as no writer is writing. In Dining Philosophers problem, one chopstick is not able to be held by more than one philosopher simultaneously.

The programs use semaphores and synchronized methods to coordinate access to the shared objects.

10.3 Deadlock

Deadlock happens when two or more threads wait for each other to release resources, so none of them can continue [2]. Deadlock can occur only when four conditions are present together: mutual exclusion, hold and wait, no preemption, and circular wait [2].

In the Dining Philosophers problem, a deadlock may happen when every philosopher picks up one chopstick and waits for another one, leaving everyone unable to continue.The semaphore-based program is designed In order to prevent such a situation, as it will release the first chopstick when it is unable to obtain the second one. The monitor-based solution ensures that the condition is satisfied before eating.

In the Readers-Writers problem, threads must always release the semaphore or lock that they hold. If they do not, other threads can be blocked forever.

10.4 Starvation

Starvation happens when a thread waits for a very long time, or forever, while other threads continue to make progress [2]. This is different from deadlock, because in starvation other threads can still make progress.

In a reader-preference Readers-Writers solution, a writer may starve if readers keep entering the shared resource. Such a writer-priority monitor would alleviate the issue by giving priority to waiting writers prior to permitting the admission of new readers. Nevertheless, the readers might have to wait for longer.

Whether starvation occurs depends on the synchronization mechanism and the scheduling policy of the program.

11. Sample Execution Trace Format

The examples below describe the kinds of events that appear in the execution traces. The exact event messages can be seen in the HTML files created by the four Java programs.

- **Dining Philosophers, semaphore:** A philosopher becomes hungry and requests the chopsticks. If both are available, the philosopher starts eating and releases the chopsticks after eating.
- **Dining Philosophers, monitor:** If a chopstick is not available, the philosopher waits. When notified, the philosopher checks the condition again before eating.
- **Readers-Writers, semaphore:** A reader starts reading and the reader count increases. A writer blocks while readers are present and continues when it can get exclusive access.
- **Readers-Writers, monitor:** A writer blocks while readers are active. Once the shared resource becomes available, the writer can access it, modify the data, and save the updated value.

12. Semaphore vs Monitor Comparison

The table below compares the two approaches as they are used in this assignment [2], [3].

Aspect	Semaphore approach	Monitor approach
Main mechanism	Semaphore acquire() and release() operations.	synchronized methods or blocks with condition waiting.
State protection	The programmer must coordinate the semaphore operations and the shared state carefully.	The monitor object keeps the shared data and the methods that control access to it together in one place.
Waiting	A thread blocks on acquire() when no permit is available.	The thread invokes wait() and examines the condition once again when it wakes up.
Waking threads	release() makes a permit available.	notifyAll() wakes the waiting threads so that they can check their conditions.
Common risk	A missing release() or inconsistent locking can stop progress.	Wrong condition checks or wrong notification logic can stop progress.
Readability	Permits are flexible, but they need careful bookkeeping.	The condition and the shared state are kept together in one object.

Semaphores also control shared resources well, but acquire() and release() have to be managed carefully.

13. Limitations and Improvements

All four programs demonstrate thread synchronization using semaphores and monitors. However, each approach has certain limitations that could be addressed to improve the programs.

- **Thread execution order:** The order in which threads run can change from one run to another. The programs may need to be run several times to see different execution orders.
- **Thread interruptions:** Thread interruptions should be handled properly so that threads can stop safely.
- **Consistent event recording:** Shared-state information should be recorded while the data is protected by the synchronization mechanism. This keeps the event logs consistent.
- **HTML safety:** Any text added to HTML or JavaScript should be properly escaped or encoded to avoid display issues and other errors.
- **Browser replay:** The HTML controls only replay events that were recorded earlier. They do not control the Java threads after execution has finished.
- **Fairness and starvation:** The algorithms should be tested to check whether any reader, writer, or philosopher can wait forever.
- **Testability and readability:** The code should be easy to read and test. If the code is changed, the programs should be tested again before the final screenshots and results are added to the report.

These improvements would make the programs more reliable and would help readers understand the synchronization through the recorded events.

14. AI Assistance Disclosure

I used ChatGPT (OpenAI) to learn about synchronization, to develop and debug the Java code, and to improve the HTML simulations of the Dining Philosophers and Readers-Writers problems. I also used it to understand semaphores, monitors, and thread synchronization.

This paper was verified using Claude (Anthropic), for checking the grammatical correctness of this report and for rephrasing technical terms.

I reviewed the suggestions from these tools and changed them where needed. I checked the program output and the technical content, and I understand the code and the concepts presented in this report.

15. Conclusion

This assignment covered two synchronization problems in Operating Systems: the Dining Philosophers problem and the Readers-Writers problem. I implemented each problem in Java using semaphores and monitors. Each program also records its execution events and generates an HTML simulation of the solution. The programs show how threads can be synchronized.

During testing, the Readers-Writers monitor program used four reader threads and two writer threads. Each thread performed three iterations.The value of the shared data went from 0 to 6, and the HTML dashboard was generated. This assignment helped me understand mutual exclusion, thread coordination, and condition checking.

16. References

[1] *Operating Systems Programming Assignment: AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation*, course assignment handout.

[2] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.

[4] E. W. Dijkstra, "Cooperating sequential processes," in *Programming Languages*, F. Genuys, Ed. London, U.K.: Academic Press, 1968, pp. 43–112.

[5] E. W. Dijkstra, "Hierarchical ordering of sequential processes," *Acta Informatica*, vol. 1, no. 2, pp. 115–138, 1971.

[6] P. J. Courtois, F. Heymans, and D. L. Parnas, "Concurrent control with 'readers' and 'writers'," *Communications of the ACM*, vol. 14, no. 10, pp. 667–668, Oct. 1971.

[7] C. A. R. Hoare, "Monitors: An operating system structuring concept," *Communications of the ACM*, vol. 17, no. 10, pp. 549–557, Oct. 1974.

[8] Oracle, *Java Platform, Standard Edition API Documentation*. Available: https://docs.oracle.com/en/java/

[9] Oracle, *The Java Tutorials: Concurrency*. Available: https://docs.oracle.com/javase/tutorial/essential/concurrency/

[10] OpenAI, ChatGPT, and Anthropic, Claude, used for programming assistance and report review, respectively. Refer to the AI Assistance Disclosure for details of their use.

17. Academic Integrity Declaration

I confirm that I have reviewed the programs and their execution results as described in this report. I have acknowledged the AI tools and external resources used while preparing this assignment. I take responsibility for explaining the synchronization concepts, the HTML simulations, and the modifications made to the code. To the best of my knowledge, I have not submitted another student's work or used third-party material without proper acknowledgment.

Name:Diya P Shetty

USN:NNM24IS080

Signature: _____

Date: _____